

GAMMA · SECURITY REVIEW

Gamma

Limit Order Manager.

A security review of Limit Order Manager.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Gamma	Limit Order Manager	Guardian	July 10 – July 18, 2025	July 29, 2025

§ 00

Contents.

30 pages — overview, confidence ranking, scope, 19 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 19 findings	P. 09
§ 05	Disclaimer	P. 29
§ 06	About Guardian	P. 30

§ 01

Overview.

Guardian reviewed Limit Order Manager across a single review round. The final report includes 2 High, 17 Low findings.

AUDIT FIRM	REPOSITORY
Guardian	GammaStrategies/algebra-limit-order
CLIENT	METHODOLOGY
Gamma	Smart contract · Manual · Main Review
PROJECT	
Limit Order Manager	
PREPARED BY	
Guardian	
AUDIT WINDOW	
July 10 – July 18, 2025	
FINAL REPORT	
July 29, 2025	

1.1 Confidence Ranking

4 / 5

High Confidence

Guardian assigns a Confidence Ranking of 4 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
 Critical	0	0	0	0	0
 High	2	0	0	0	2
 Medium	0	0	0	0	0
 Low	17	0	3	0	14
 Informational	0	0	0	0	0
All severities	19	0	3	0	16

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence ASSIGNED	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Limit Order Manager in `GammaStrategies/algebra-limit-order`.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	Missing External Pause/Unpause Functions	Access Control	HIGH	RESOLVED	p. 9
H-02	Potential Fee Theft Via Minting	Rewards	HIGH	RESOLVED	p. 10
L-01	Unused Variable <code>limitOrderPluginAddr</code>	Best Practices	LOW	RESOLVED	p. 11
L-02	Unused <code>onlyKeeper</code> Modifier	Best Practices	LOW	RESOLVED	p. 12
L-03	Duplicate Validation Call In Limit Orders	Best Practices	LOW	RESOLVED	p. 13
L-04	Repeated Encoding Of Callback Data	Best Practices	LOW	RESOLVED	p. 14
L-05	Unused Errors	Informational	LOW	RESOLVED	p. 15
L-06	Risks Of Native ETH Handling	Warning	LOW	RESOLVED	p. 16
L-07	Inconsistent Reentrancy Protection	Validation	LOW	RESOLVED	p. 18
L-08	Redundant Updates In <code>_claimOrder</code>	Gas Optimization	LOW	RESOLVED	p. 19
L-09	Temporary Denial Of Service: Order Creation	Warning	LOW	ACKNOWLEDGED	p. 20
L-10	Uninitialised Config Params	Validation	LOW	ACKNOWLEDGED	p. 21
L-11	Cancel Flow Skips Nonce Update, Unlike Execute Path	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 22
L-12	Unnecessary Type Casting	Informational	LOW	RESOLVED	p. 23

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-13	Inconsistency Risk Due To Mutable Tick Spacing	Warning	LOW	RESOLVED	p. 24
L-14	Exposure To Read-Only Reentrancy	Logical Error	LOW	RESOLVED	p. 25
L-15	Dust Can Be Lost In Claim Flow	Rounding	LOW	RESOLVED	p. 26
L-16	Unused Imports	Informational	LOW	RESOLVED	p. 27
L-17	sizeSkew Is Uncapped	Best Practices	LOW	RESOLVED	p. 28

HIGH **H-01**

RESOLVED

Missing External Pause/Unpause Functions

CATEGORY Access Control

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L25-L26>

REVIEW Main Review

DESCRIPTION

Gamma has added `Pausable` to the Limit Order Manager with the intention of pausing order creation when necessary. However, the contract does not expose any external functions to actually pause or unpause it.

`Pausable` only provides internal functions `_pause()` and `_unpause()` and expects the implementing contract to wrap them in access-controlled public or external functions.

```
function _pause() internal virtual whenNotPaused {
    _paused = true;
    emit Paused(_msgSender());
}

function _unpause() internal virtual whenPaused {
    _paused = false;
    emit Unpaused(_msgSender());
}
```

RECOMMENDATION

Add access-controlled external functions to allow authorized parties (e.g., the admin) to pause and unpause the contract.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb>

HIGH **H-02**

RESOLVED

Potential Fee Theft Via Minting

CATEGORY	Rewards
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

A malicious actor can mint a small position directly on the Algebra pool with the `recipient` set to the `LimitOrderManager` contract. This triggers a `feeGrowth` update for the position (keyed by `top`, `bottom`, `recipient`) during the `mint` call.

Later, when `LimitOrderManager` invokes `retrackPositionFee`, it uses the updated `feeGrowthInside` values, effectively skipping over the previously accumulated fees — thereby denying them to the legitimate order owners.

The same vulnerability exists on Gamma's Uniswap v4 Limit Order Manager as well.

RECOMMENDATION

Consider allowing addition of liquidity only if recipient is `msg.sender` of the call via `beforeModifyPosition` hook.

PROOF OF CONCEPT <https://gist.github.com/GuardianAudits/082274ec940166015e7dc89d0f16f1e2>

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-dda20f096ebcc273759d674522ed164dcc15da6bfb7dce07c2982021e4bb8bc>

LOW **L-01**

RESOLVED

Unused Variable `limitOrderPluginAddr`

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L48-L49>

REVIEW Main Review

DESCRIPTION

The variable `limitOrderPluginAddr` is declared and is admin-writable, but it is never read in the contract. The same value can be fetched directly using `pool.plugin()`.

RECOMMENDATION

Consider removing `limitOrderPluginAddr` and its corresponding setter to reduce unnecessary storage and simplify the contract.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-02**

RESOLVED

Unused onlyKeeper Modifier

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L68-L69>

REVIEW Main Review

DESCRIPTION

The `onlyKeeper` modifier is defined in the contract but is never used. Instead, Gamma uses `require(msg.sender == keeper, "...")` directly wherever the keeper check is needed.

RECOMMENDATION

Either remove the unused `onlyKeeper` modifier to reduce dead code, or refactor the contract to consistently use the modifier instead of repeating the `require` check.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-03**

RESOLVED

Duplicate Validation Call In Limit Orders

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L153>

REVIEW Main Review

DESCRIPTION

The function `validateScaleOrderSizes()` is invoked even for limit orders. The function checks both the first and last order's `amount` against `minRequired`, which results in the same value being checked twice when `orders.length == 1` (limit orders).

```
if (orders[0].amount < minRequired) {  
    revert MinimumAmountNotMet(orders[0].amount, minRequired);  
}  
if (orders[orders.length - 1].amount < minRequired) {  
    revert MinimumAmountNotMet(orders[orders.length - 1].amount, minRequired);  
}
```

RECOMMENDATION

Consider doing second validation only when `orders.length > 1`

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-843c029c40654e821f4bd1ec1b635b945f543e0a386e9c6caa17d53899d0e4fc>

LOW **L-04**

RESOLVED

Repeated Encoding Of Callback Data

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L162>

REVIEW Main Review

DESCRIPTION

Gamma currently encodes `bytes memory callbackData = abi.encode(msg.sender);` within each loop iteration, despite the encoded data being the same each time.

RECOMMENDATION

Define `callbackData` once before the loop and reuse it to avoid redundant encoding operations.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-05**

RESOLVED

Unused Errors

CATEGORY	Informational
LOCATION	TickLibrary.sol
REVIEW	Main Review

DESCRIPTION

The `WrongTickRange`, `InvalidPrice`, and `PriceMustBeGreaterThanZero` errors in `TickLibrary` are not used anywhere in the codebase and can be removed.

RECOMMENDATION

Remove unused errors.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-e84461201fb9fea9b8eb5f750bb28d2bd55527838531078c6e565a000608d151>

LOW **L-06**

RESOLVED

Risks Of Native ETH Handling

CATEGORY	Warning
LOCATION	LimitOrderManager.sol
REVIEW	Main Review

DESCRIPTION

`LimitOrderManager` accepts and handles native ETH.

- Mint: Excess ETH is refunded immediately upon receipt.
- Claim: ETH is transferred back to the user.

However, ETH transfers hand over control to the user during the call, which can introduce subtle reentrancy paths or cause inconsistent state views.

Example – Mint:

1. `createLimitOrder` is called with excess ETH.
2. Ticks are validated such that both `bottomTick` and `topTick` are either above or below the current tick.
3. Excess ETH is refunded.
4. User reenters the pool and performs a swap moving the tick from `bottomTick` → `currentTick` → `topTick`.
5. This causes the mint to pull both tokens, instead of just one.

While this does not currently result in an exploit (since the contract doesn't hold tokens and they are pulled directly from users), it creates non-obvious and undesirable edge behavior.

RECOMMENDATION

- Ideally, remove native ETH support altogether and rely solely on WETH. This eliminates callback exposure and standardizes handling. An external wrapper can be used for ETH ↔ WETH conversion.
- Alternatively, at minimum, remove the refund logic in the mint path. Since the refund occurs before critical state changes, it introduces unnecessary risk.

Instead, enforce that `msg.value` must exactly match the required amount.

L-06 Risks Of Native ETH Handling

CONTINUED

— RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-07**

RESOLVED

Inconsistent Reentrancy Protection

CATEGORY Validation
LOCATION LimitOrderManager.sol
REVIEW Main Review

DESCRIPTION

Some public/external functions in `LimitOrderManager` lack the `nonReentrant` modifier, despite similar functions being protected.

For example:

1. `cancelPositionKeys` is guarded with `nonReentrant`, whereas `cancelBatchOrder` is not.
2. Inside `cancelOrder`, during `_handlePositionRemoval`, before the user is removed from `positionContributors`, and `positionState` is changed, if there is an ETH callback in `claim`, one can reenter and use the unaltered state.

There are multiple such cases where one can reenter `LimitOrderManager` and use the stale or to be updated state.

While we haven't been able to exploit these cases profitably, we feel that allowing a set of possibilities where there is no benefit to the user is not ideal, since it leads to unnecessary exposure.

RECOMMENDATION

Consider applying `nonReentrant` to all user-facing actions.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-08**

RESOLVED

Redundant Updates In `_claimOrder`

CATEGORY Gas Optimization

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L520>

REVIEW Main Review

DESCRIPTION

The `_claimOrder` function updates the user's position. However the position is deleted right after. This is redundant and wastes a lot of gas.

RECOMMENDATION

Consider not updating the state here, rely on the variables already in memory and be aware to not read from the not updated state variable.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-09**

ACKNOWLEDGED

Temporary Denial Of Service: Order Creation

CATEGORY Warning
LOCATION LimitOrderManager.sol
REVIEW Main Review

— DESCRIPTION

A large swap can mark many tick ranges as `waitingForKeeper`. While these ranges are queued for keeper execution, new orders that overlap these ticks cannot be placed. This behavior—though intentional—can block order creation across a wide range of ticks, creating a temporary denial-of-service condition for users and integrators.

This mechanism can be unintuitive to third-party integrators or bots that expect open access to place limit orders at any time.

— RECOMMENDATION

No changes to core logic are required. However, consider documenting this behavior explicitly.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Uninitialised Config Params

CATEGORY	Validation
LOCATION	https://github.com/GuardianOrg/algebra-limit-order-gamma/limitordermanager-team1/blob/main/src/LimitOrderManager.sol
REVIEW	Main Review

DESCRIPTION

The `minAmount0` and `minAmount1` params are zero right after deploying the `LimitOrderManager` contract and the admin needs to set them manually with the `setMinAmount` later on.

This might allow malicious actors to back run the deployment and create smaller positions than the admin would like to allow, which could lead to bad consequences.

RECOMMENDATION

Consider setting the parameters in the constructor or define hardcoded default value for them.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Cancel Flow Skips Nonce Update, Unlike Execute Path

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L417-L418
REVIEW	Main Review

DESCRIPTION

In the `handlePositionRemoval` case of a cancel—where `positionContributors`'s length is zero—the `feePerLiquidity` for the position is not reset, and the nonce is not updated.

As a result, if a user creates an order, cancels it, and then creates it again with the same range, the `feePerLiquidity` from the previous position persists and is reused in the next `createOrder` for that range.

While we haven't been able to exploit this (since the logic is additive), this behavior is inconsistent—especially considering the cancel flow mimics `execute` (`isActive = false`, `isWaitingForKeeper = false`, `removePositionFromTick`).

This inconsistency could lead to issues down the line.

RECOMMENDATION

Consider updating the nonce whenever `positionContributors`'s length is zero in cancel.

RESOLUTION

Status: Acknowledged.

LOW **L-12**

RESOLVED

Unnecessary Type Casting

CATEGORY	Informational
LOCATION	https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L401
REVIEW	Main Review

DESCRIPTION

On line 401 of the LimitOrderManager contract, the pool variable is cast to the IAlgebraPool interface.

```
IAlgebraPool(pool).burn(bottomTick, topTick, uint128(userLiquidity), "");
```

However, `pool` is already an `IAlgebraPool`, and the cast is unnecessary.

RECOMMENDATION

Consider removing unnecessary casting

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-13**

RESOLVED

Inconsistency Risk Due To Mutable Tick Spacing

CATEGORY Warning
LOCATION Global
REVIEW Main Review

DESCRIPTION

Algebra allows pool owners to modify `tickSpacing` via the `setTickSpacing` function. However, Gamma's `LimitOrderManager` treats tick spacing as immutable and relies on it for all range-based calculations. If `tickSpacing` is changed on an Algebra pool after deployment, it will result in inconsistent and potentially incorrect behavior across Gamma's logic, including fee calculation, position placement, and execution.

RECOMMENDATION

Although Algebra permits `tickSpacing` updates, such changes must not be performed on any pools associated with Gamma. This constraint should be explicitly documented for both internal developers and external integrators.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/7b1435e55591ae2bcc4c95f616c67b413ea56d48>

LOW **L-14**

RESOLVED

Exposure To Read-Only Reentrancy

CATEGORY	Logical Error
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

Gamma reads Algebra pool state directly from global storage slots (e.g., `globalState()`), which exposes it to read-only reentrancy attacks. Since Algebra uses locking to guard state consistency during critical operations, bypassing these locks by reading global state directly may lead to edge case inconsistencies — especially if exploited by another contract in the same transaction context.

— RECOMMENDATION

Replace direct calls to `globalState()` with `safelyGetStateOfAMM`, which respects the pool's internal locking mechanism. If additional state is needed beyond what `safelyGetStateOfAMM` provides, **first verify pool state via `isUnlocked()`** before proceeding with reads.

— RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commits/mitigations/>

LOW **L-15**

RESOLVED

Dust Can Be Lost In Claim Flow

CATEGORY	Rounding
LOCATION	https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/LimitOrderManager.sol#L525-L551
REVIEW	Main Review

— DESCRIPTION

In the `_claimOrder` a full share price calculation is performed twice to figure out the amount of fees that belong to the user and to the treasury.

As both round down this could leave dust tokens stuck inside the contract.

— RECOMMENDATION

Consider working with subtraction instead to be more precise.

— RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba49>

LOW **L-16**

RESOLVED

Unused Imports

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The following imports are not used in the codebase and can be removed:

- `console.sol` and `Constants.sol` in `LimitOrderManager`
- `console.sol` in `PositionManagement`
- `{console}` from `Test.sol` and `IAAlgebraVirtualPool.sol` in `LimitOrderPlugin`

RECOMMENDATION

Consider removing unused imports.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-387a5e54fe14d0e9709790a25aff03a67f1130b343dcf71b99dcd3be9dd4ba9>

LOW **L-17**

RESOLVED

sizeSkew Is Uncapped

CATEGORY Best Practices

LOCATION <https://github.com/GuardianOrg/algebra-limit-order-gammalimitordermanager-team1/blob/3064a1f9e8f03dc6e6c93818cb518c0d7822024f/src/PositionManagement.sol#L76>

REVIEW Main Review

DESCRIPTION

Users can provide any `sizeSkew` param when calling the `createScaleOrders` function except from 0. A very big `sizeSkew` value leads to a buffer overflow Dos in the `denominator1` calculation, in the `_calculateOrderSize` function.

RECOMMENDATION

Consider capping the `sizeSkew` param.

RESOLUTION

Status: Resolved.

<https://github.com/GammaStrategies/algebra-limit-order/commit/55c3734576ff05ce1e73c6ca059c2f5cbf7d9bcb#diff-843c029c40654e821f4bd1ec1b635b945f543e0a386e9c6caa17d53899d0e4fc>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.